

quent itemsets. In the second step, the frequent itemsets are used to discover the association rules.

Apriori algorithm is perhaps the most well-known algorithm in the data mining field for finding frequent itemsets. A key issue in the Apriori algorithm is the performance of the subset function. A fast way of determining "which candidate itemsets are involved in a transaction" is needed. This can be achieved by storing the candidate itemsets in a structure called 'hash tree.' All nodes are initially created as leaf nodes but can later be converted into internal nodes if they contain too many itemsets. Internal nodes are hash tables where each bucket points to another node. In the root node, the branch to follow is decided by applying a hashing function to the first (lowest numbered) item in the itemset.

At the next level of the tree, the hashing function is applied to the second function and so on. There are many improvements made on the existing Apriori algorithm where huge databases are involved.

One possible improvement is to keep track of the itemsets by book-keeping in each transaction in the first pass itself by counting the 1-itemsets (itemsets containing one item). Using this method, the size of the structure is smaller than the actual database. Thus a hybrid plan can be used in the early pass and switched to alternative method if the bookkeeping structure becomes too huge as there is a linear relationship with the number of transactions in the Apriori method. However, there is a risk of itemsets getting left out if they don't occur frequently as the algorithm is run on randomly selected transactions. A lower threshold on the confidence and support could be used to reduce the risk.

Its drawback

The Apriori algorithm, however, suffers from some drawbacks:

1. It may need to generate a huge number of candidate sets, for example. If there are 100,000 frequent

1-itemsets, the Apriori algorithm will need to generate more than 1,000,000 candidate 2-itemsets and accumulate and test the occurrence frequencies. Moreover, to discover a frequency pattern of size 100, such as a1, a2, a3, ..., a100, it must generate more than 10^{30} candidates in total.

2. It may need to repeatedly scan the database and check the last set of candidates by pattern matching. This is especially the case for mining long patterns.

Solution: the tree approach

I have adopted a tree-based approach over the Apriori algorithm. This tree-based structure enables faster scanning and quicker analysis.

The T-tree differs from more standard set enumeration trees in that the nodes at the same level in any sub-branch are organised into 1-D arrays so that array indices represent column numbers.

For this purpose, we make a reverse tree. This permits direct indexing with the attribute/column numbers. The T-tree offers two initial advantages over standard set enumeration trees:

1. Since indexing mechanism is used, there is fast traversal of the tree.
2. The requirement of pointers is avoided as explicit storage of the reduced itemset labels is not desired.

Strategy: Vertical partitioning

The partitioning algorithm commences by distributing the input dataset over the available number of processes using a vertical partitioning strategy. Initially, the set of single attributes (columns) is split equally between the available processes so that an allocation itemset is defined for each process in terms of a start column number and end column number.

Each process will have its own allocation itemset, which is then used to determine the subset of the input dataset to be considered by the pro-

cess. Using its allocated itemset, each process proceeds as follows:

1. Remove all records in the input dataset that do not have common interest with the allocation itemset.
2. From the remaining records, remove those attributes whose column number is greater than the end column number. We cannot remove those attributes whose identifiers are less than the start column number because they may represent the sub-string of a large itemset to be included in the subset counted by the process.

The process begins with a top-level tree comprising only those 1-itemsets included in its allocation itemset. The process then generates the candidate 2-itemsets that belong to its sub T-tree. These will comprise all the possible pairings between each element in the allocation itemset of those elements. The support values for the candidate 2-itemsets are then determined and the sets then pruned to leave only large 2-itemsets. Thus in this way, third-level candidate sets can be generated. No attributes from succeeding partitions need be considered. However, the subsets involved in preceding an allocation of itemset will be present in the possible candidate itemsets, which would be counted by some other process.

Result: Faster execution

In terms of the execution time, the vertical partitioning algorithm performs much better than the task distribution algorithms because of the messaging overhead. Also, for distributed algorithm, as processes are added, the increasing overhead of messaging more than outweighs any gain from using additional processes, so distribution/parallelisation becomes counterproductive. The partitioning shows some gain from the addition of further processes. Hence vertical partitioning gives the best results and best scaling. ●

The author is an assistant professor in MCA department of SCMS School of Technology & Management (SSTM), Cochin, Kerala